

Tổng Hợp Kiến Thức

1. Bối Cảnh Ra Đời - Tại Sao Cần Event Loop?

1.1 Giới thiệu Node.js

- **Node.js**: Runtime JavaScript phía server, dùng **V8 Engine** (Chrome).
- Ứng dụng: Xử lý nhiều kết nối đồng thời, đặc biệt với tác vụ I/O (file, mạng, database).

1.2 Blocking I/O (Truyền thống)

- **Định nghĩa**: Chương trình **dừng lại chờ** I/O hoàn thành.
- **Ví dụ ẩn dụ**: Nhân viên thu ngân chỉ phục vụ 1 khách, đợi khách tìm hàng xong mới tiếp tục.
- **Hạn chế**:
 - Giảm hiệu năng khi xử lý nhiều yêu cầu.
 - Lãng phí CPU (rảnh rỗi khi chờ I/O).

1.3 Non-Blocking I/O (Node.js)

- **Định nghĩa**: Gửi yêu cầu I/O, chương trình **tiếp tục chạy ngay**, callback xử lý kết quả sau khi I/O xong.
- **Ví dụ ẩn dụ**: Nhân viên giao việc cho bếp, tiếp tục phục vụ khách khác, bếp báo lại khi xong.
- **Ưu điểm**:
 - Xử lý nhiều yêu cầu đồng thời.
 - Tận dụng CPU hiệu quả.

1.4 Tại Sao Cần Event Loop?

- **Vấn đề**: JavaScript **single-threaded**, cần cơ chế điều phối tác vụ bất đồng bộ.
- **Giải pháp**: **Event Loop** quản lý việc:
 - Khi nào I/O hoàn thành?
 - Thực thi callback nào, theo thứ tự gì?
- **Vai trò**: Đảm bảo xử lý non-blocking I/O trong môi trường single-threaded.

2. Các Thành Phần Cốt Lõi của Event Loop

2.1 JavaScript Single-Threaded

- Chỉ **một đoạn mã JavaScript** chạy tại một thời điểm.
- Non-blocking I/O được xử lý bởi **Node APIs** và **Event Loop**, không phải JavaScript trực tiếp.

2.2 Call Stack (Ngăn xếp cuộc gọi)

- **Cơ chế**: LIFO (Last-In, First-Out).
- **Chức năng**: Theo dõi và thực thi các hàm JavaScript.
- **Hoạt động**:
 - Hàm gọi → đẩy frame vào đỉnh stack.
 - Hàm hoàn thành (return) → pop frame ra.
- **Ví dụ**:**Luồng Call Stack**:

```
console.log('a');
function b() { console.log('b'); }
function c() { console.log('c'); b(); }
c();
```

1. `console.log('a')` → in "a" → ra.
2. `c()` → vào.
3. `console.log('c')` → in "c" → ra.
4. `b()` → vào.
5. `console.log('b')` → in "b" → ra.
6. `b()` → ra.
7. `c()` → ra.

Kết quả: `a` , `c` , `b` .

2.3 Node APIs / C++ APIs

- **Chức năng:** Xử lý tác vụ I/O (file, mạng, timers) ở **background** (qua libuv hoặc OS).
- **Ví dụ:** `fs.readFile` , `setTimeout` , `http.request` .
- **Hoạt động:**
 - Gọi hàm bất đồng bộ → giao việc cho Node APIs.
 - APIs xử lý (thường trên background threads) → đưa callback vào Callback Queue khi xong.
- **Lưu ý:** Không chạy trên Call Stack, giúp JavaScript tiếp tục thực thi.

2.4 Callback Queue (Task Queue)

- **Cơ chế:** FIFO (First-In, First-Out).
- **Chức năng:** Lưu các callback đã sẵn sàng (sau khi I/O hoặc timer hoàn thành).
- **Hoạt động:**
 - Callback được đẩy vào **cuối Queue** khi tác vụ bất đồng bộ xong.
 - Chờ Event Loop đưa vào Call Stack.
- **Lưu ý:** Không thực thi ngay, chỉ chờ lượt.

3. Event Loop - Người Điều Phối

3.1 Nhiệm vụ

- **Chạy liên tục**, kiểm tra:
 - **Call Stack** có rỗng không?
 - **Callback Queue** có callback nào không?
- **Nếu Call Stack rỗng:**
 - Lấy callback đầu tiên từ Queue (FIFO).
 - Đẩy vào Call Stack để thực thi.
- **Nếu Call Stack không rỗng:**
 - Chờ, không làm gì.

3.2 Ví dụ với `setTimeout`

```
console.log("Bắt đầu");
setTimeout(() => { console.log("Callback setTimeout 0ms"); }, 0);
setTimeout(() => { console.log("Callback setTimeout 100ms"); }, 100);
console.log("Kết thúc");
```

Luồng hoạt động:

1. `console.log("Bắt đầu")` → Call Stack → in → ra.
2. `setTimeout(..., 0)` → Call Stack → giao timer cho Node APIs → ra.

- 3. `setTimeout(..., 100)` → Call Stack → giao timer cho Node APIs → ra.
- 4. `console.log("Kết thúc")` → Call Stack → in → ra.
- 5. (~0ms) Timer 0ms xong → callback1 vào Callback Queue.
- 6. Event Loop: Call Stack rỗng → đẩy callback1 vào Call Stack → in "Callback setTimeout 0ms".
- 7. (~100ms) Timer 100ms xong → callback2 vào Callback Queue.
- 8. Event Loop: Call Stack rỗng → đẩy callback2 vào Call Stack → in "Callback setTimeout 100ms".

Kết quả: `Bắt đầu` , `Kết thúc` , `Callback setTimeout 0ms` , `Callback setTimeout 100ms` .

3.3 Ví dụ với `fs.readFile`

```
const fs = require('fs');
console.log("Bắt đầu đọc file...");
fs.readFile('myFile.txt', 'utf8', (err, data) => {
  if (err) { console.error("Lỗi đọc file:", err); return; }
  console.log("Đọc file thành công!");
});
console.log("Yêu cầu đọc file đã được gửi đi.");
```

Luồng hoạt động:

- 1. `console.log("Bắt đầu đọc file...")` → Call Stack → in → ra.
- 2. `fs.readFile` → Call Stack → giao đọc file cho Node APIs → ra.
- 3. `console.log("Yêu cầu đọc file đã được gửi đi.")` → Call Stack → in → ra.
- 4. File đọc xong → callback vào Callback Queue.
- 5. Event Loop: Call Stack rỗng → đẩy callback vào Call Stack → in "Đọc file thành công!".

Kết quả: `Bắt đầu đọc file...` , `Yêu cầu đọc file đã được gửi đi.` , `Đọc file thành công!` .

4. Lưu ý Quan trọng

- **Call Stack rỗng** là điều kiện để Event Loop xử lý callback.
- Code **đồng bộ** luôn chạy trước callback bất đồng bộ, dù delay là 0ms (vì callback phải qua Callback Queue).
- **Node APIs** xử lý I/O ở background, không block Call Stack.
- **Event Loop** chỉ điều phối, không thực thi I/O hay callback.